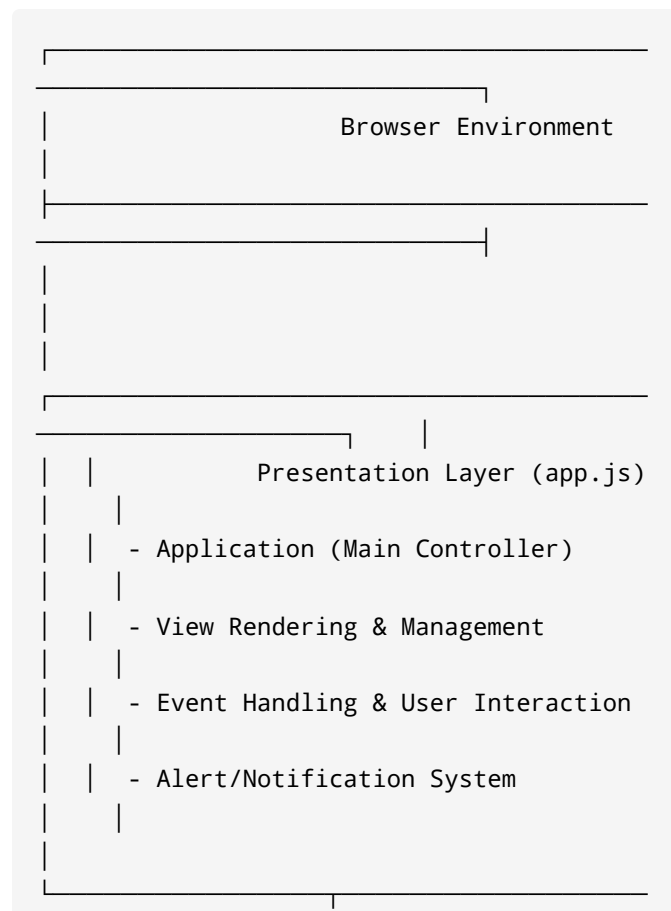
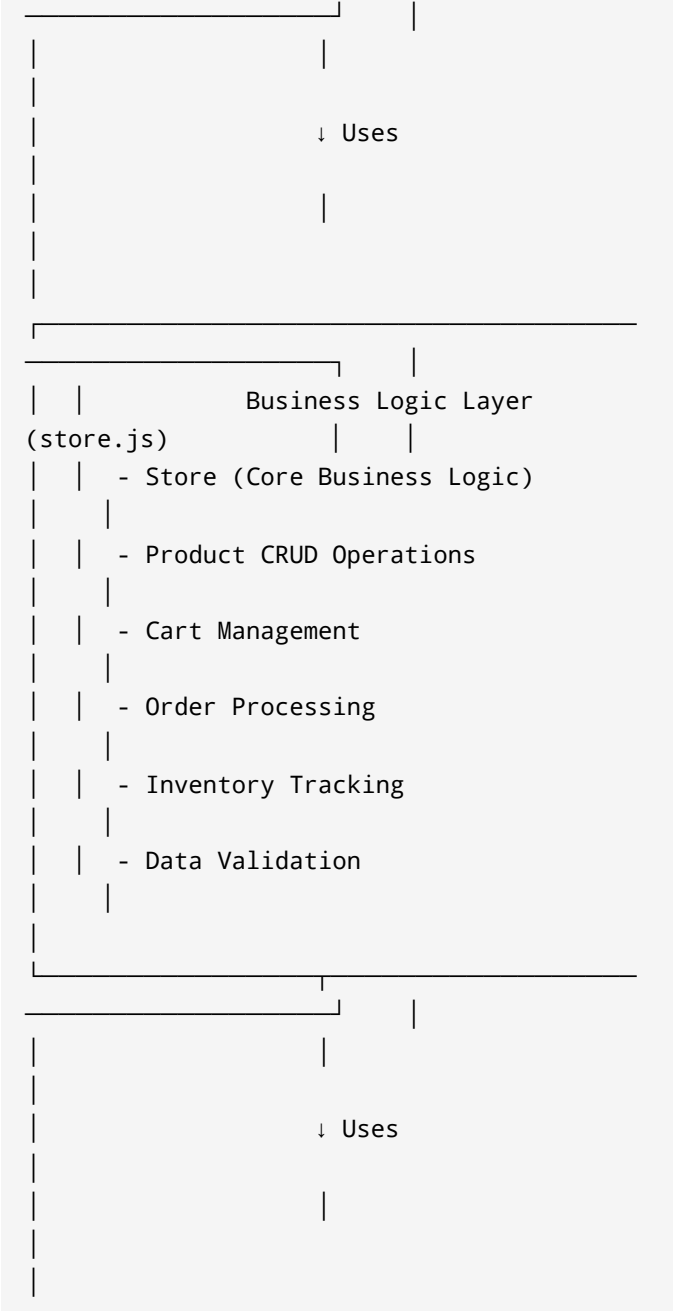
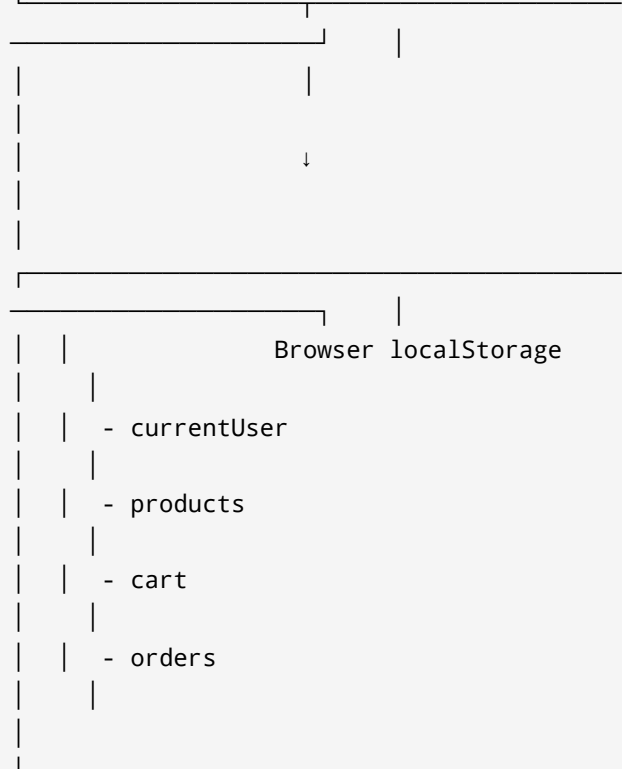
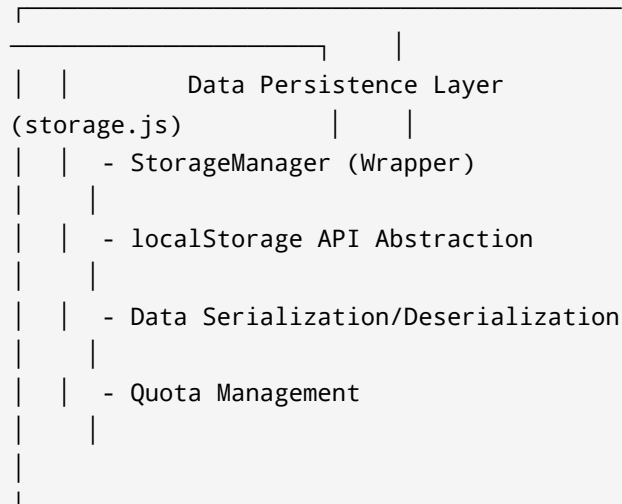


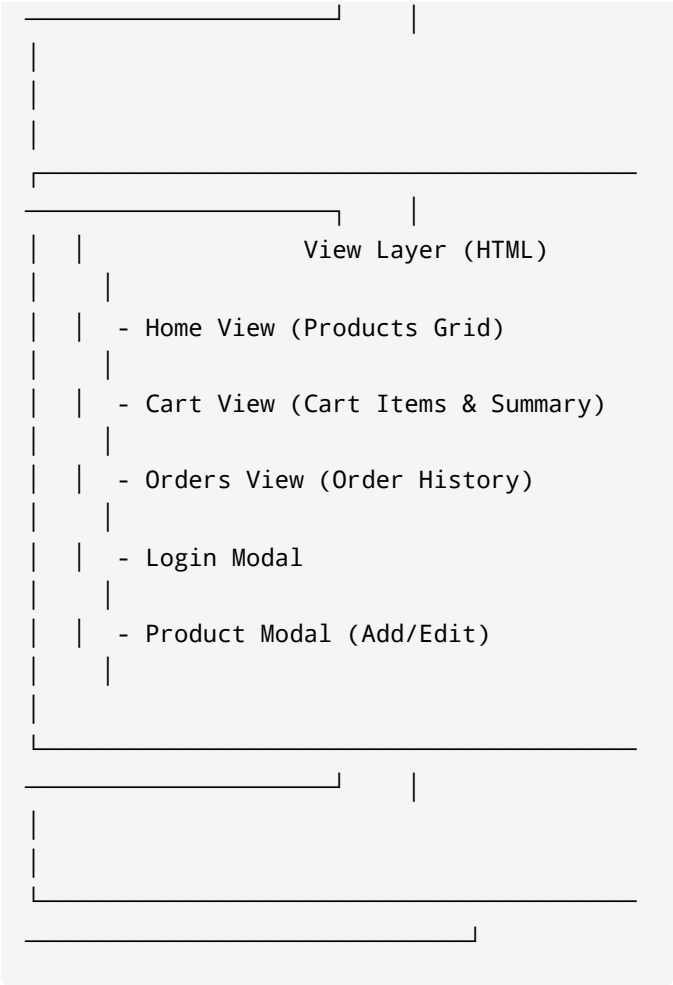
EcoShop Capstone - Architecture Documentation

System Architecture Overview









Module Breakdown

1. Storage Layer (storage.js)

Purpose: Abstract browser localStorage into a clean, type-safe interface

Class: `StorageManager`

Key Methods:

```
set(key, value)           // Save to
localStorage              // Retrieve
get(key, defaultValue)    // Delete from
from localStorage         // Clear all
remove(key)               // Get storage
clear()                   statistics

// Convenience methods
saveUser(user)
getUser()
saveProducts(products)
getProducts()
saveCart(cart)
getCart()
saveOrders(orders)
getOrders()
initializeSampleData()    // Initialize
with demo data
```

Data Format:

```
localStorage[ecoshop_currentUser] =
JSON.stringify({ id, username, email, ...
})
localStorage[ecoshop_products] =
JSON.stringify([{id, name, price, stock,
...}])
localStorage[ecoshop_cart] =
JSON.stringify([{productId, quantity,
...}])
localStorage[ecoshop_orders] =
JSON.stringify([{id, items, total, status,
...}])
```

Error Handling:

- Catches JSON parse errors
 - Handles QuotaExceededError gracefully
 - Returns default values on failures
-

2. Business Logic Layer (store.js)

Purpose: Implement core application logic and data management

Class: Store

State:

```
this.products    // Array of all products
this.cart        // Array of cart items
                 (references productId)
```

```
this.orders          // Array of completed
orders
this.nextProductId // Counter for product
IDs
this.nextOrderId   // Counter for order IDs
```

Product CRUD Operations

Create:

```
addProduct(product) → newProduct
// Input: { name, description, price,
stock, emoji }
// Output: { id, name, description, ...,
createdAt }
// Action: Adds to array, increments ID
counter, persists to storage
```

Read:

```
getProducts()      → Array
getProduct(id)     → Product | undefined
```

Update:

```
updateProduct(id, updates) →
updatedProduct
// Input: id, { fields to update }
// Output: Updated product object
// Action: Merges updates, adds updatedAt,
persists
```

Delete:

```
deleteProduct(id) → deletedProduct
// Output: Deleted product
// Action: Removes from array, persists to
storage
```

Cart Operations

Add to Cart:

```
addToCart(productId, quantity) →
updatedCart
// Validation: Product exists, quantity <=
stock
// Logic:
//   - If product already in cart: increase
quantity
//   - Otherwise: create new cart item
// Persists: saveCart()
```

Update Quantity:

```
updateCartQuantity(productId, quantity) →
updatedCart
// Validation: quantity > 0, quantity <=
stock
// If quantity = 0: removeFromCart()
```

Get Cart with Details:

```
getCartWithDetails() → Array
// Maps cart items to include full product
data
// Calculates subtotal per item
// Returns: [{ productId, quantity,
product, subtotal }]
```

Calculate Total:

```
getCartTotal() → number
// Returns: Sum of all subtotals
```

Order Operations

Create Order:

```
createOrder(customerInfo) → order
// Pre-condition: cart.length > 0
// Logic:
// 1. Create order object with items
// 2. Calculate subtotal, tax (10%),
total
// 3. Deduct stock from each product
// 4. Add order to orders array
// 5. Clear cart
// 6. Persist all changes
// Returns: { id, items, subtotal, tax,
total, status, ... }
```

Cancel Order:

```
cancelOrder(orderId) → order
// Pre-condition: order.status !==
'cancelled'
// Logic:
// 1. Restore stock for all items
// 2. Set status to 'cancelled'
// 3. Persist changes
```

3. Presentation Layer (app.js)

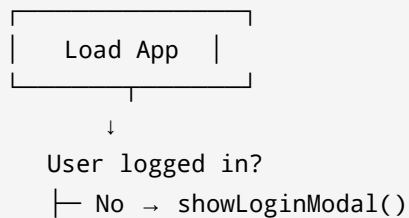
Purpose: Handle UI rendering and user interactions

Class: Application

State:

```
this.currentUser      // Logged-in user
this.currentView      // Current visible
view ('home'|'cart'|'orders')
this.editingProductId // ID of product
being edited (null if creating)
```

Authentication Flow



```

└─ Yes → hideLoginModal() →
updateHeader() → showHome()
    ↓
login(event)
├─ Validate username & password
├─ Create user object
├─ storage.saveUser(user)
├─ hideLoginModal()
├─ updateHeader()
└─ showAlert("Welcome, ${username}!")

```

View Management

```

switchView(viewName)    // Hide all views,
show specific one
showHome()              // Show products
showCart()              // Show cart items
& summary
showOrders()            // Show order
history

```

View Elements:

```

#homeView      → Products grid + Add
button
#cartView      → Cart items + Summary
#ordersView    → Order list
#loginModal    → Login form
#productModal  → Add/Edit product form

```

Product Management Flow

Add Product:

```
showAddProduct()  
├─ Clear form fields  
├─ Set modal title to "Add Product"  
├─ Show modal  
│  
User submits form  
│  
saveProduct(event)  
├─ Prevent default  
├─ Get form values  
├─ Validate inputs  
├─ store.addProduct(productData)  
├─ closeProductModal()  
├─ renderProducts()  
└─ showAlert("Product added")
```

Edit Product:

```
showEditProduct(productId)  
├─ Get product from store  
├─ Populate form with product data  
├─ Set modal title to "Edit Product"  
├─ Store productId in  
  this.editingProductId  
├─ Show modal  
│  
User submits form  
│  
saveProduct(event)  
├─ Check if editingProductId exists  
├─ If yes: store.updateProduct()
```

```
└─ If no: store.addProduct()  
└─ ... (same as add)
```

Delete Product:

```
deleteProduct(productId)  
└─ Confirm with user  
└─ store.deleteProduct(productId)  
└─ renderProducts()  
└─ showAlert("Product deleted")
```

Cart Management Flow

Add to Cart:

```
addToCart(productId)  
└─ store.addToCart(productId, 1)  
└─ updateCartBadge() → Badge shows count  
└─ showAlert("Added to cart")
```

Cart Rendering:

```
renderCart()  
└─ Get store.getCartWithDetails()  
└─ If empty: show empty state  
└─ If items:  
|   └─ Map items to HTML cards  
|   └─ Add quantity +/- buttons  
|   └─ Add remove buttons  
|   └─ updateCartSummary()
```

Checkout:

```
checkout()  
├ store.createOrder(customerInfo)  
├ updateCartBadge()  
├ renderCart() → Shows empty cart  
├ showAlert("Order #${id} placed")  
└ setTimeout(() => showOrders(), 500)
```

Data Flow Diagrams

Adding a Product to Cart

```
User clicks "Add to Cart"  
↓  
addToCart(productId)  
↓  
store.addToCart(productId, 1)  
├ Get product  
├ Validate stock  
├ Check if in cart  
│ ├ Yes: increment quantity  
│ └ No: create new item  
├ Add/update in cart array  
└ storage.saveCart(cart)  
↓  
updateCartBadge()  
├ Get cart.length  
├ Update DOM badge  
└ Set badge.style.display
```

↓
`showAlert("Added to cart")`

Placing an Order

```
User clicks "Checkout"
  ↓
  checkout()
    ↓
    store.createOrder(customerInfo)
      ├── Validate cart not empty
      ├── Create order object
      ├── For each cart item:
      |   ├── Get product
      |   ├── Deduct stock
      |   ├── Update product
      |   └── storage.saveProducts()
      ├── Add order to orders array
      ├── storage.saveOrders()
      └── storage.saveCart([])
    ↓
    updateCartBadge() → Badge becomes 0
    ↓
    renderCart() → Shows empty cart
    ↓
    showAlert("Order #1001 placed!")
    ↓
    setTimeout(() => showOrders(), 500)
    ↓
    renderOrders() → Shows new order
```

State Consistency

Products ↔ Cart ↔ Orders Sync

Products Update

```
↓
store.updateProduct(id, updates)
  └─ Update in products array
  └─ storage.saveProducts()
    └─ Check cart for product
      └─ If editing price: reflect in
cart totals
      └─ If editing stock: validate cart
quantities
```

Cart Update

```
↓
store.addToCart() / removeFromCart() /
updateCartQuantity()
  └─ Validate against current product
stock
  └─ Update cart array
  └─ storage.saveCart()
```

Order Creation

```
↓
store.createOrder()
  └─ Lock current prices/products
  └─ Deduct stock from products
  └─ storage.saveProducts()
  └─ Add order to orders
  └─ storage.saveOrders()
```

```
└─ Clear cart
  └─ storage.saveCart([])
```

Design Patterns

1. Singleton Pattern

```
// storage.js
const storage = new StorageManager(); //
Single instance

// store.js
const store = new Store(); // Single
instance

// app.js
window.app = new Application(); // Single
instance
```

2. MVC Pattern

- **Model:** `store.js` (data & business logic)
- **View:** `capstone-index.html` (DOM & layout)
- **Controller:** `app.js` (orchestrates M & V)

3. Facade Pattern

- `storage.js` abstracts localStorage complexity
- `store.js` abstracts business logic complexity

4. Observer Pattern (simplified)

- Event listeners in `app.js` observe DOM events
- UI updates reactively to data changes

5. Lazy Loading

- Products rendered only when needed
 - No pre-rendering of all views
-

Error Handling Strategy

Try-Catch Blocks

```
try {
  store.addToCart(productId);
  this.updateCartBadge();
  this.showAlert('Added to cart',
    'success');
} catch (error) {
  this.showAlert(`Error:
    ${error.message}`, 'error');
}
```

Validation

```
// At store level
if (quantity > product.stock) {
  throw new Error('Insufficient stock.
```

```
Available: ${product.stock}`);
}

// At app level
if (!productData.name ||
!productData.description) {
    this.showAlert('Please fill in all
fields', 'error');
    return;
}

// User confirmation
if (confirm('Are you sure?')) {
    // Proceed
}
```

Storage Errors

```
set(key, value) {
    try {
        localStorage.setItem(fullKey,
JSON.stringify(value));
        return true;
    } catch (error) {
        if (error.name ===
'QuotaExceededError') {
            // Storage full
        }
        return false;
    }
}
```

Performance Optimizations

1. Minimal Reflows

- Batch DOM updates
- Use innerHTML for lists instead of appendChild

2. Efficient Event Delegation (where applicable)

- Single listener for multiple elements
- Event bubbling handled

3. localStorage Efficiency

- Minimal JSON stringify/parse operations
- Data read once at initialization

4. No External Dependencies

- Pure vanilla JavaScript
- No library overhead

5. CSS Optimization

- Hardware acceleration (transform, opacity)
 - Will-change sparingly
 - Media queries for responsive breaks
-

Testing Strategy

Unit Testing (at store level)

```
// Add product
const product = store.addProduct({ name:
"Test", price: 10, ... });
assert(product.id > 0);
assert(store.getProduct(product.id) !==
undefined);

// Cart operations
store.addToCart(productId, 2);
assert(store.cart[0].quantity === 2);

// Order creation
const order =
store.createOrder(customerInfo);
assert(order.id > 0);
assert(store.cart.length === 0); // Cart
cleared
```

Integration Testing

```
// Full checkout flow
store.addProduct({ name: "A", price: 10,
stock: 5, ... });
store.addToCart(1, 2);
const order =
store.createOrder(customerInfo);
const product = store.getProduct(1);
```

```
assert(product.stock === 3); // Stock
deducted
assert(order.items[0].productName === "A");
```

UI Testing

- Click add to cart → Badge updates
- Submit form → Modal closes, list updates
- Logout → Login modal shows
- Responsive test → No horizontal scroll at 320px

Security Architecture

Client-Side XSS Prevention

```
// Unsafe
element.innerHTML = userInput; // ✖

// Safe
element.textContent = userInput; // ✔
escapeHtml(userInput);           // ✔
```

Input Validation

```
// HTML
<input type="number" min="0" max="999" />
<input type="email" />
```

```
// JavaScript
if (!productData.name ||
!productData.description) {
    throw new Error('Required field
missing');
}
```

CORS & Origin

```
// All data stored locally
// No external API calls to untrusted
sources
// No sensitive data transmitted
```

Scalability Considerations

Current Limitations

- **Storage Limit:** ~5-10MB per domain (varies by browser)
- **Performance:** 1000+ products may slow UI
- **No Backend:** All data in-memory/localStorage

Future Scaling

1. Backend API

- Move to server-side database

- Add authentication API
- Implement real payment processing

2. **Caching Strategy**

- Implement IndexedDB for larger datasets
- Add service workers for offline mode

3. **Code Splitting**

- Separate modules into individual files
- Import only needed code

4. **Database**

- PostgreSQL/MongoDB
- Proper indexing
- Query optimization

Deployment Checklist

- ☐ Code review completed
- ☐ All tests passing
- ☐ No console errors
- ☐ No console warnings
- ☐ Performance audit done
- ☐ Lighthouse score > 90
- ☐ Mobile responsive verified

- ☐ Cross-browser tested
 - ☐ Documentation complete
 - ☐ README updated
 - ☐ .gitignore configured
 - ☐ Push to GitHub
 - ☐ Deploy to Vercel/Netlify
 - ☐ Live URL verified
 - ☐ Links in submission form
-

Architecture Version: 1.0

Last Updated: September 24, 2026